

Equations and Math Supplement

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Equations and Math Supplement (Appendix)	1
1.1 Volume of a Tetrahedron (Lattice)	1
1.2 Expanded Ace 5×5 Matrix	2
1.3 Cayley-Menger Determinant (Coxeter.4D)	2
1.4 Piero della Francesca Formula (PdF)	3
1.5 Gerald de Jong Formula (GdJ)	3
1.6 Fisher Information Matrix (FIM)	3
1.7 Empirical Fisher Information Matrix	4
1.8 Natural Gradient	4
1.9 Free Energy (Active Inference)	4
1.10 Expected Free Energy (Active Inference)	5
1.11 Quadray Normalization (Fuller.4D)	5
1.12 Distance (Embedding Sketch; Coxeter.4D slice)	5
1.13 Minkowski Line Element (Einstein.4D analogy)	5
1.14 High-Precision Arithmetic Note	5
1.14.1 Reproducibility artifacts and external validation	5
1.15 Namespaces summary (notation)	6

1 Equations and Math Supplement (Appendix)

Conventions for this appendix: vertices are $P_0, \dots, P_3$ with Quadray components (a_i, b_i, c_i, d_i) ; volumes are V_{xyz} (Euclidean, cubic length units) and V_{ivm} (synergetics/IVM tetravolume units, unit regular tetrahedron $V_{ivm} = 1$), related by $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Section 3); M denotes the Quadray-to-XYZ embedding matrix (Sections 3 and 14), $\|\cdot\|_2$ the Euclidean norm, and $\text{KL}[Q \| P]$ the Kullback-Leibler divergence of Q relative to P (Section 10). Three symbol overloads are scoped by context and flagged where they occur: c is a Quadray component (and a PdF edge length) in the volume formulas but the speed of light in the Minkowski line element; d is the fourth Quadray component in the coordinate formulas but an edge length in the length-based formulas and the distance function $d(\cdot, \cdot)$ of the embedding section; and P is a vertex label P_i in the volume sections but a probability distribution in the free-energy sections.

1.1 Volume of a Tetrahedron (Lattice)

$$V_{xyz} = \frac{1}{6} |\det [P_1 - P_0, P_2 - P_0, P_3 - P_0]| \quad (1)$$

Notes.

- $P_0, \dots, P_3$ are Cartesian (XYZ) vertex coordinates, in length units; each bracketed column is an edge vector, the determinant is the volume of the parallelepiped they span, and the $1/6$ factor converts it to the tetrahedron volume V_{xyz} in cubic length units. This is the coordinate (difference) form of the homogeneous-row determinant of Eq. (4); for IVM units directly from Quadray coordinates, see the native formula of Eq. (7).

Tom Ace 5×5 tetravolume (IVM units):

$$V_{ivm} = \frac{1}{4} \left| \det \begin{pmatrix} a_0 & b_0 & c_0 & d_0 & 1 \\ a_1 & b_1 & c_1 & d_1 & 1 \\ a_2 & b_2 & c_2 & d_2 & 1 \\ a_3 & b_3 & c_3 & d_3 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix} \right| \quad (2)$$

Notes.

- Row i lists the four Quadray components (a_i, b_i, c_i, d_i) of vertex P_i , augmented with an affine 1; the last row $(1, 1, 1, 1, 0)$ encodes the projective normalization constraint, so the determinant is invariant to adding (t, t, t, t) to every vertex (Section 3). Division by 4 returns the IVM tetravolume V_{ivm} ; for integer quadrays the determinant is an exact integer (Bareiss algorithm), computed as a `fractions.Fraction`. This matrix is identical, row for row, to the named matrix Q of Eq. (3).

1.2 Expanded Ace 5×5 Matrix

The Ace 5×5 matrix of Eq. (2) written out explicitly and named Q :

$$Q(P_0, P_1, P_2, P_3) = \begin{bmatrix} a_0 & b_0 & c_0 & d_0 & 1 \\ a_1 & b_1 & c_1 & d_1 & 1 \\ a_2 & b_2 & c_2 & d_2 & 1 \\ a_3 & b_3 & c_3 & d_3 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}, \quad V_{ivm} = \frac{1}{4} |\det Q(P_0, \dots, P_3)| \quad (3)$$

Notes.

- **Matrix structure:** row i holds the four Quadray components (a_i, b_i, c_i, d_i) of vertex P_i , plus the affine coordinate 1 — literally the same rows as Eq. (2).
- **Last row:** $(1, 1, 1, 1, 0)$ enforces the projective normalization constraint (Section 3).
- **Volume computation:** $V_{ivm} = \frac{1}{4} |\det Q|$ in IVM units, exact for integer quadrays.
- **Notation:** the Ace matrix is written Q ; the symbol M is reserved for the Quadray-to-XYZ embedding matrix (Sections 3 and 14).

XYZ determinant volume and S3 conversion:

$$V_{xyz} = \frac{1}{6} \left| \det \begin{pmatrix} x_0 & y_0 & z_0 & 1 \\ x_1 & y_1 & z_1 & 1 \\ x_2 & y_2 & z_2 & 1 \\ x_3 & y_3 & z_3 & 1 \end{pmatrix} \right|, \quad V_{ivm} = S3 V_{xyz}, \quad S3 = \sqrt{\frac{9}{8}} \quad (4)$$

Notes.

- Homogeneous-row determinant in Cartesian coordinates: each row is $(x_i, y_i, z_i, 1)$ for vertex P_i , with coordinates in length units; the absolute determinant divided by 6 is the Euclidean volume V_{xyz} in cubic length units. This is the homogeneous form of Eq. (1); conversion to IVM units uses $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ as used throughout.

1.3 Cayley-Menger Determinant (Coxeter.4D)

For tetrahedron volume from edge lengths (Coxeter.4D approach):

$$288 V_{xyz}^2 = \det \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & d_{01}^2 & d_{02}^2 & d_{03}^2 \\ 1 & d_{10}^2 & 0 & d_{12}^2 & d_{13}^2 \\ 1 & d_{20}^2 & d_{21}^2 & 0 & d_{23}^2 \\ 1 & d_{30}^2 & d_{31}^2 & d_{32}^2 & 0 \end{pmatrix} \quad (5)$$

Notes.

- **Pairwise distances:** d_{ij} is the Euclidean distance between vertices P_i and P_j in length units; the matrix stores the squared distances d_{ij}^2 , so the formula is length-only — no coordinates enter (Coxeter.4D).
- **Length-only formulation:** Cayley-Menger provides a length-only formula for simplex volumes, here specialized to tetrahedra.
- **Conversion to IVM:** V_{xyz} is the Euclidean volume in cubic length units; use $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Eq. (4)); the PdF formula of Eq. (6) consumes the same lengths in closed algebraic form.

1.4 Piero della Francesca Formula (PdF)

For tetrahedron volume from edge lengths meeting at a vertex:

$$144 V_{xyz}^2 = 4a^2b^2c^2 - a^2(b^2+c^2-f^2)^2 - b^2(c^2+a^2-e^2)^2 - c^2(a^2+b^2-d^2)^2 + (b^2+c^2-f^2)(c^2+a^2-e^2)(a^2+b^2-d^2) \quad (6)$$

Notes.

- **Edge lengths:** a, b, c are the three edges meeting at the apex vertex P_0 — $a = d_{01}$, $b = d_{02}$, $c = d_{03}$ — and d, e, f are the respective opposite edges, $d = d_{23}$, $e = d_{13}$, $f = d_{12}$, in length units (the same d_{ij} as Eq. (5)).
- **Conversion to IVM:** V_{xyz} is the Euclidean volume in cubic length units (the prefactor 144 is the standard Heron-like PdF normalization); use $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Eq. (4)).

1.5 Gerald de Jong Formula (GdJ)

Native Quadray formula for tetrahedron volume:

$$V_{ivm} = \frac{1}{4} |\det [\pi(P_1) - \pi(P_0), \pi(P_2) - \pi(P_0), \pi(P_3) - \pi(P_0)]|, \quad \pi(P_i) = (a_i - d_i, b_i - d_i, c_i - d_i) \quad (7)$$

Notes.

- **Projection:** π maps a Quadray vertex P_i with components (a_i, b_i, c_i, d_i) to $\mathbb{R}^3$ by subtracting the fourth component from the first three; each bracketed column is a projected edge vector. Because π kills the direction $(1, 1, 1, 1)$, it is invariant under $q \rightarrow q + t(1, 1, 1, 1)$ — the projective fiber of Eq. (??) (Section 14) — so the determinant is unchanged by projective normalization of the vertices. When all four vertices share the same fourth component (in particular $d_i = 0$), $\pi(P_i)$ reduces to (a_i, b_i, c_i) .
- **Native IVM:** no S3 conversion; the factor $1/4$ returns IVM tetravolume directly — for the unit tetrahedron (origin plus three IVM neighbor moves) the determinant is exactly 4, giving $V_{ivm} = 1$ (Section 3).
- **Exact arithmetic:** integer Quadray coordinates give an integer determinant, evaluated exactly by the Bareiss algorithm as a `fractions.Fraction`; the code implements Eq. (7) as `integer_tetra_volume`, and the magnitude agrees exactly with the Ace determinant of Eq. (2) on every integer-quadray input.

See code: `tetra_volume_cayley_menger`. For tetrahedron volume background, see [Tetrahedron - volume](#). Exact integer determinants in code use the [Bareiss algorithm](#). External validation: these formulas align with implementations in the 4dsolutions ecosystem. See the [Resources](#) section for comprehensive details.

1.6 Fisher Information Matrix (FIM)

Background: [Fisher information](#).

$$F_{i,j} = \mathbb{E} \left[\frac{\partial \log p(x; \theta)}{\partial \theta_i} \frac{\partial \log p(x; \theta)}{\partial \theta_j} \right] \quad (8)$$

Notes.

- **Symbols:** x is an observation, $\theta = (\theta_1, \dots, \theta_m)$ the parameter vector, and $p(x; \theta)$ the likelihood; the expectation is over $x \sim p(\cdot; \theta)$, so F is a function of θ . Each score $\partial \log p(x; \theta) / \partial \theta_i$ carries inverse parameter units, so $F_{i,j}$ carries inverse squared parameter units per observation.
- F is symmetric positive semi-definite: large eigenvalues mark sensitive (high-curvature) directions, small eigenvalues sloppy ones (Section 4 and the Discussion). The empirical estimate of Eq. (9) is the per-observation counterpart.
- See code: `fisher_information_matrix` in `src/quadmath/inference/information.py` — empirical outer-product estimator. Figure: the empirical estimate is shown in the FIM heatmap figure of Section 4.

1.7 Empirical Fisher Information Matrix

For empirical estimation from data, the Fisher Information Matrix is computed as:

$$F_{i,j} = \frac{1}{N} \sum_{n=1}^N \frac{\partial \log p(x_n; \theta)}{\partial \theta_i} \frac{\partial \log p(x_n; \theta)}{\partial \theta_j} \quad (9)$$

Notes.

- **Symbols:** $x_1, \dots, x_N$ are N i.i.d. observations and $g_n = \nabla_{\theta} \log p(x_n; \theta)$ the per-sample score vector (same units as Eq. (8)); the estimator is $F = \frac{1}{N} \sum_n g_n g_n^T$, with a `normalize` flag controlling the $1/N$ division.
- Converges to Eq. (8) as $N \rightarrow \infty$; used by natural-gradient descent (Eq. (10)) and the information-geometry applications of Section 4.

1.8 Natural Gradient

Background: [Natural gradient](#) (Amari).

$$\theta \leftarrow \theta - \eta F(\theta)^{-1} \nabla_{\theta} L(\theta) \quad (10)$$

Explanation.

- **Symbols:** θ the parameter vector, $L(\theta)$ a differentiable loss, $\nabla_{\theta} L$ its gradient, $F(\theta)$ the Fisher matrix of Eq. (8) — or its empirical estimate, Eq. (9) — evaluated at θ , and $\eta > 0$ the step size (learning rate).
- **Update:** right-preconditioning by the inverse Fisher metric gives the steepest-descent direction under the Fisher metric (Amari), invariant to smooth invertible reparameterizations of θ .
- **Damping:** the implementation `natural_gradient_step` solves the damped system $(F + \lambda I) \delta = \nabla_{\theta} L$ and applies $\theta \leftarrow \theta - \eta \delta$, with a small Tikhonov ridge λ (default 10^{-9}) keeping the solve stable when F is near-singular.

See code: `natural_gradient_step` in `src/quadmath/inference/information.py` — damped inverse-Fisher step.

1.9 Free Energy (Active Inference)

$$\mathcal{F} = -\log P(o \mid s) + \text{KL}[Q(s) \parallel P(s)] \quad (11)$$

Explanation.

- **Symbols:** o the observation, s the latent state, $Q(s)$ the approximate posterior (recognition distribution), and P the generative model, so $P(o \mid s)$ is the likelihood and $P(s)$ the prior; $\mathcal{F}$ is the variational free energy in nats (natural logarithm, Section 10). **Notation flag:** capital P here is a probability distribution, unrelated to the vertex labels P_i of the volume sections.
- **Partition:** minimizing $\mathcal{F}$ over Q trades the expected negative log-likelihood $\mathbb{E}_Q[-\log P(o \mid s)]$ (the displayed $-\log P(o \mid s)$ is read under this Q -expectation, as implemented) against the KL divergence $\text{KL}[Q \parallel P]$ that ties the posterior to the prior; see [Free energy principle](#).

See code: `free_energy` in `src/quadmath/inference/information.py` — discrete-state variational free energy (inputs are unnormalized distributions, normalized internally).

Note: The main figures demonstrating natural gradient trajectories and free energy landscapes are shown in [Section 4: Optimization in 4D](#). The appendix focuses on unique figures specific to mathematical formulations and validation.

1.10 Expected Free Energy (Active Inference)

Background: [Active Inference \(Parr, Pezzulo & Friston, MIT Press, 2022\)](#).

$$G = \text{KL}[Q(s) \parallel P(s)] - \mathbb{E}_q[\log P(o \mid s)] - \log P(o) \quad (12)$$

Explanation.

- **Symbols:** $Q(s)$ the approximate posterior and $P(s)$ the prior over states (as in Eq. (11)), $P(o \mid s)$ the likelihood, $P(o)$ the prior preference over outcomes in nats (uniform when omitted), $H[Q] = -\sum_s Q(s) \log Q(s)$ the Shannon entropy in nats, and $\mathbb{E}_q$ expectation under Q ; G is the expected free energy minimized during action selection.
- **Epistemic term:** the KL divergence between variational posterior and prior over states.
- **Entropy:** the posterior entropy is already inside the KL term, since $\text{KL}[Q \parallel P] = \mathbb{E}_q[\log Q(s)] - \mathbb{E}_q[\log P(s)] = -H[Q(s)] - \mathbb{E}_q[\log P(s)]$; subtracting H again would count it twice.
- **Ambiguity:** the negative expected log-likelihood of outcomes penalizes noisy observations.
- **Pragmatic term:** prior preferences $P(o)$ enter negatively ($-\log P(o)$), so preferred outcomes lower G ; agents minimize G during action selection.

See code: `expected_free_energy` in `src/quadmath/inference/information.py` — all four terms of Eq. (12) as implemented.

1.11 Quadray Normalization (Fuller.4D)

Given a Quadray $q = (a, b, c, d)$, choose $k = \min(a, b, c, d)$ and set $q' = q - (k, k, k, k)$, enforcing non-negative entries with at least one zero. The shift is the projective normalization of Section 3 — q and q' represent the same direction, the fiber formalized in Eq. (??) (Section 14). Here k is the normalization offset of the glossary (Section 10); it is unrelated to the shell-frequency k of the lattice sections (Sections 11 and 13).

1.12 Distance (Embedding Sketch; Coxeter.4D slice)

Choose a linear map M from Quadray space to $\mathbb{R}^3$ (or $\mathbb{R}^4$) consistent with the tetrahedral axes — the embedding matrix of Sections 3 and 14 (canonical Urner family, Eq. (??)). Then for Quadray points p, q , the distance is $d(p, q) = \|M(p - q)\|_2$ in length units; under the integer Urner family the squared distance is the exact integer identity of Eq. (??), which the lattice-search layer ranks with (Section 13).

1.13 Minkowski Line Element (Einstein.4D analogy)

$$ds^2 = -c^2 dt^2 + dx^2 + dy^2 + dz^2 \quad (13)$$

Background: [Minkowski space](#).

Signature convention $(-, +, +, +)$ (mostly-plus, Section 2): c is the speed of light, (t, x, y, z) are spacetime coordinates in time and length units respectively, and ds^2 has units of length squared. Here c is the physical constant, not the Quadray component or Pdf edge length used in the volume sections.

1.14 High-Precision Arithmetic Note

When evaluating determinants, FIMs, or geodesic distances for sensitive problems, use quad precision (binary128) via GCC's `libquadmath` (`_float128`, functions like `expq`, `sqrtq`, and `quadmath_snprintf`). See [GCC libquadmath](#). Where possible, it is useful to use symbolic math libraries like SymPy to compute exact values.

1.14.1 Reproducibility artifacts and external validation

- **This manuscript's artifacts:** Raw data in `quadmath/output/` for reproducibility and downstream analysis:
 - `fisher_information_matrix.csv / .npz`: empirical Fisher matrix and inputs
 - `fisher_information_eigenvalues.csv / fisher_information_eigensystem.npz`: eigenspectrum and eigenvectors
 - `natural_gradient_path.png` with `natural_gradient_path.csv / .npz`: projected trajectory and raw coordinates
 - `ivm_neighbors_data.csv / ivm_neighbors_edges_data.npz`: neighbor coordinates (Quadray and XYZ)
 - `polyhedra_quadray_constructions.png`: synergetics volume relationships schematic
- **External validation resources:** The [4dsolutions ecosystem](#) provides extensive cross-validation. See the [Resources](#) section for comprehensive details on computational implementations and validation.

1.15 Namespaces summary (notation)

- `Coxeter.4D`: Euclidean E^4 ; regular polytopes; not spacetime (cf. Coxeter, *Regular Polytopes*, Dover ed., p. 119). Connections to higher-dimensional lattices and packings as in Conway & Sloane.
- `Einstein.4D`: Minkowski spacetime; indefinite metric; used here only as a metric analogy when discussing geodesics and information geometry.
- `Fuller.4D`: Quadrays/IVM; tetrahedral lattice with integer tetravolume; unit regular tetrahedron has volume 1; synergetics scale relations (e.g., S3).